🎵 MusicGraph

Exploration des collaborations musicales via NoSQL Graphe

Lucas Diot — *Tech Talk (30 min)*

Neo4j 5

TypeScript

React & vis-network

MusicBrainz API

Docker

🎯 1. La Problématique : La musique est un réseau

🎶 Le monde musical

- Morceaux créés en **featurings** et duos.
- Écosystème de labels, genres et pays.
- Questions naturelles :
  - *"Lien le plus court Angèle ↔ Booba ?"*
  - *"Artiste pivot d'une scène musicale ?"*

🛑 La limite du SQL

- Multiplications de jointures $N \times M$.
- Requêtes récursives complexes (`WITH RECURSIVE`).
- **Temps dégradé** avec la profondeur d'exploration (N sauts).

2. Pourquoi Neo4j & le Modèle Graphe ?

Le paradigme : *Index-Free Adjacency*

- Chaque nœud stocke en mémoire des **pointeurs directs vers ses voisins**.
- Traverser une relation se fait en temps constant $O(1)$ par saut.

Base Relationnelle (SQL)

- Scan de tables d'index à chaque jointure.
- Le temps dépend du **volume total de la base**.

Base Graphe (Neo4j)

- Navigation directe par pointeurs mémoire.
- Le temps dépend uniquement du **voisinage local**.

📐 3. Modélisation du Graphe (Property Graph)

Nœuds (Entités)

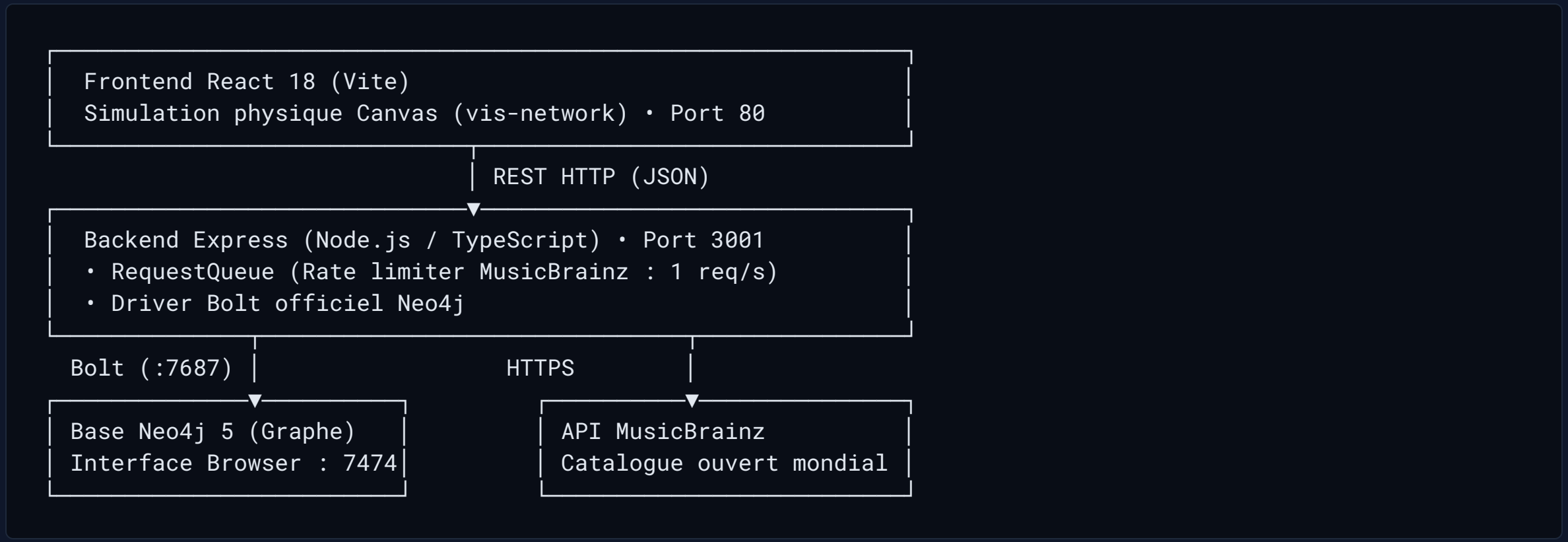
- `(:Artist)` : Nom, pays, dates, MBID
- `(:Recording)` : Morceaux & singles
- `(:Release)` : Albums & EPs
- `(:Label)` : Maison de disques
- `(:Genre)` & `(:Area)` : Style & Pays

Relations typées

- `(:Artist)-[:PERFORMED]->(:Recording)`
  `(:Artist)-[:FEATURED_ON]->`
- `(:Recording)`
  `(:Recording)-[:APPEARS_ON]->`
- `(:Release)`
  `(:Artist)-[:COLLABORATED_WITH]->`
- `(:Artist)`

(avec attribut `{count}` précalculé)

4. Architecture Globale



5. Défi 1 : Le Rate-Limit MusicBrainz

La contrainte externe

- API MusicBrainz : **1 requête par seconde** maximum par IP.
- Dépassement = ban temporaire (**HTTP 429 / 503**).

La solution : `RequestQueue` personnalisée

- **File d'attente FIFO asynchrone** traitant les requêtes séquentiellement.
- Espacement garanti de **1 100 ms** entre chaque appel.
- Retries avec backoff exponentiel en cas de saturation.
- **Catalogue mock (** `DEMO_CATALOG` **)** en fallback garanti si coupure réseau.

🔄 6. Défi 2 : Idempotence & Nœuds Squelettes

1. Dédoublonnage universel avec `MERGE` (Cypher)

Chaque entité est identifiée de façon unique par son **MBID** :

```
MERGE (a:Artist {mbid: $mbid})
ON CREATE SET a.name = $name, a.lastImported = datetime()
ON MATCH SET a.lastImported = datetime()
```

2. Les "Nœuds Squelettes" & le Crawler

- L'import d'un artiste crée ses collaborateurs en nœuds légers.
- Un **Crawler de fond** permet d'enrichir récursivement les collaborateurs sans saturer l'API.

💻 7. Live Démo — L'Application Web

Démonstration interactive en direct sur `http://localhost` *:*

- **Dashboard & Métriques** :
  - Volumes en direct (artistes, morceaux, albums, collaborations).
- **Statistiques Réseau** :
  - **Centralité de degré** : artistes les plus connectés (ex: *PLK*).
  - **Top collaborations** : duos les plus fréquents.
- **Exploration Visuelle** :
  - Navigation dans le graphe Canvas avec moteur physique (**ForceAtlas2**).
  - Filtrage à la volée des types de nœuds.

⚡ 8. Live Démo — Sous le capot (Neo4j Browser)

Requêtes Cypher directes sur `http://localhost:7474` :

◆ Le plus court chemin (*Kevin Bacon Effect*)

```
MATCH (p1:Artist {name: "PLK"}), (p2:Artist),
      path = shortestPath((p1)-[:COLLABORATED_WITH*]-(p2))
WHERE p1 <> p2
RETURN path LIMIT 1
```

◆ Sous-graphe des collaborations directes

```
MATCH (a1:Artist)-[c:COLLABORATED_WITH]-(a2:Artist)
RETURN a1, c, a2 LIMIT 40
```

🎨 9. Performance Frontend : Canvas vs React State

⚠️ Le piège classique en React

- Gérer les coordonnées (x, y) de 400 nœuds dans le state (`useState`).
- Résultat : **60 re-renders / seconde** $\rightarrow$ freeze du navigateur.

💡 Notre architecture

- Conteneur DOM détaché piloté via `useRef` .
- Simulation physique et tracé Canvas **100% délégués à `vis-network` .**
- React ne traite que les événements métier (sélection, filtres).

🚀 10. Bilan & Scalabilité

🏆 Acquis du projet

- Modèle de graphe expressif et performant.
- Pipeline robuste face aux contraintes d'API.
- Déploiement reproductible via Docker Compose.

🔮 Évolutions possibles

- **Neo4j Graph Data Science (GDS)** :
  - Détection de communautés (Louvain).
  - Score d'autorité (PageRank).
- Enrichissement audio via Spotify API.

❓ Questions & Réponses

Merci pour votre écoute !

- **Frontend Web** : http://localhost
- **Neo4j Browser** : http://localhost:7474
- **Stack** : Neo4j • Node.js / Express • TypeScript • React • Docker